

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

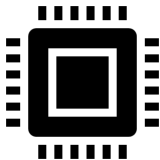
División de Ciencias de la ingeniería

Ingeniería

Estructura de Datos Sección “A”

Ing. Mario Moises Ramirez Tobar

Primer Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

TAD'S Practica #1



Descripción:

En el siguiente documento se le presentaron todos los TAD 'S utilizados para poder realizar la primera práctica del curso de estructura de datos.

TAD'S PRINCIPALES DE LA JERARQUÍA DE CLASES DE CARTAS

Estos son la parte principal del juego ya que permiten representar los modelos y comportamientos de cada carta que tiene uso en el juego.

Modelo

Interfaz Pública:
Modelo(const ColorCarta&_colorRef, const std::string &_nombre,const std::string &_lado, TipoCarta _tipo, int _jerarquia)
void setColor(const ColorCarta &_color)
void setNombre(const std::string &_nombre)
void setLado(const std::string &_lado)
std::string getNombre()const
ColorCarta getColor()const
std::string getLado()const
TipoCarta getTipo()const
int getJerarquia()const
std::string getReversoModelo()const
virtual std::string getPathImagen() const = 0
virtual void lanzarCarta(ListaCircular<Jugador*> & jugador) = 0
virtual void lanzarCarta(Partida & partidaActual) = 0
virtual void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) = 0

Implementación Privada:
std::string nombre
ColorCarta color
int jerarquia
std::string lado
TipoCarta tipo

Debido a que la clase **Modelo** es la clase padre de una gran jerarquia de cartas aca se presentan los TAD'S de cada una de las clases hijas.

Bloqueo

Interfaz Pública:
Bloqueo(const ColorCarta&_colorRef, const std::string &_nombre,const std::string &_lado,int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
~Bloqueo()
METODOS HEREDADOS DE Modelo

Implementación Privada:
ATRIBUTOS HEREDADOS DE Modelo

CambioDireccion

Interfaz Pública:
CambioDireccion(const ColorCarta& _colorRef, const std::string & _nombre, const std::string & _lado, int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
~CambioDireccion()
METODOS HEREDADOS DE Modelo

Implementación Privada:
ATRIBUTOS HEREDADOS DE Modelo

ColorEterno

Interfaz Pública:
ColorEterno(const ColorCarta& _colorRef, const std::string & _nombre, const std::string & _lado, int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
~ColorEterno()
METODOS HEREDADOS DE Modelo

Implementación Privada:
ATRIBUTOS HEREDADOS DE Modelo

ComodinColor

Interfaz Pública:
ComodinColor(const ColorCarta&_colorRef, const std::string &_nombre,const std::string &_lado,int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> &jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> &jugador) override
~ComodinColor()
METODOS HEREDADOS DE Modelo

Implementación Privada:
ATRIBUTOS HEREDADOS DE Modelo

Eclipse

Interfaz Pública:
Eclipse(const ColorCarta&_colorRef, const std::string &_nombre,const std::string &_lado,int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> &jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> &jugador) override
~Eclipse()
METODOS HEREDADOS DE Modelo

Implementación Privada:
ATRIBUTOS HEREDADOS DE Modelo

Espia

Interfaz Pública:
Espia(const ColorCarta& _colorRef, const std::string & _nombre,const std::string & _lado,int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
~Espia()
METODOS HEREDADOS DE Modelo

Implementación Privada:
ATRIBUTOS HEREDADOS DE Modelo

MultiColorSuma

Interfaz Pública:
MultiColorSuma(const ColorCarta& _colorRef, const std::string & _nombre, const std::string & _lado, int _jerarquia, int _valor)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
~MultiColorSuma()
METODOS HEREDADOS DE Modelo

Implementación Privada:
int valorSuma
int getValorSuma()
ATRIBUTOS HEREDADOS DE Modelo

Numerica

Interfaz Pública:
Numerica(const ColorCarta& _colorRef, const std::string & _nombre, const std::string & _lado, int _valor, int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
int getValor() const
~Numerica()
METODOS HEREDADOS DE Modelo

Implementación Privada:
int valor
std::string getDirectoriosRosa() const
std::string getDirectoriosVioleta() const
std::string getDirectoriosVioleta() const
std::string getDirectoriosTurquesa() const
std::string getDirectoriosNaranja() const
std::string getDirectoriosRojo() const
std::string getDirectoriosAzul() const
std::string getDirectoriosAmarillo() const
std::string getDirectoriosVerde() const
ATRIBUTOS HEREDADOS DE Modelo

Reversa

Interfaz Pública:
Reversa(const ColorCarta& _colorRef, const std::string & _nombre,const std::string & _lado,int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
~Reversa()
METODOS HEREDADOS DE Modelo

Implementación Privada:
ATRIBUTOS HEREDADOS DE Modelo

SaltoTodos

Interfaz Pública:
SaltoTodos(const ColorCarta& _colorRef, const std::string & _nombre,const std::string & _lado,int _jerarquia)
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
~SaltoTodos()
METODOS HEREDADOS DE Modelo

Implementación Privada:
ATRIBUTOS HEREDADOS DE Modelo

SumaCantidad

Interfaz Pública:
SumaCantidad(const ColorCarta&_colorRef, const std::string &_nombre,const std::string &_lado,int _jerarquia, int _valor)
int getValorSuma()
std::string getPathImagen() const override
void lanzarCarta(ListaCircular<Jugador*> & jugador) override
void lanzarCarta(Partida & partidaActual) override
void lanzarCarta(Partida & partidaActual, ListaCircular<Jugador*> & jugador) override
~SumaCantidad()
METODOS HEREDADOS DE Modelo

Implementación Privada:
int valorSuma
ATRIBUTOS HEREDADOS DE Modelo

TAD'S DE LAS CLASES DE ESTRUCTURAS DINÁMICAS

Estas son las estructuras de datos que se implementaron en el proyecto para poder tener una gestión de listas, pilas y listas circulares que permiten manejar las referencias de las cartas y son la lógica principal al juego.

Pila

Esta fue una estructura genérica que permite manejar las pilas de cartas tanto laterales como centrales que se manejan en el proyecto.

Interfaz Pública:
Pila()
int getLongitud() const
bool estaVacia() const
void push(const T& valor)
void pop()
T& verTop()
~Pila()

Implementación Privada:
Nodo<T> * cima
int longitud

ListaCircular

Esta fue una estructura genérica que permite manejar los jugadores y sobre todo las rondas que se manejan en cada turno de la partida.

Interfaz Pública:
ListaCircular()
bool esVacia()const
int getLongitud()const
void insertar(const T& valor)
void eliminarEn(int indice)
void avanzar()
void retroceder()
T& getActual()
T& pickSiguiente()
T& pickAnterior()
T& obtenerEn(int indice)
T obtenerConstanteEn(int indice) const
~ListaCircular()

Implementación Privada:
Nodo<T> * cabeza
Nodo<T> * actualTurno
int longitud
Nodo<T> * buscarNodo(int posicion) const

ListaEnlazada

Esta fue una estructura genérica que permite manejar los mazos que cada jugador posee al igual que fueron utilizadas para poder manejar el orden y desorden de las cartas.

Interfaz Pública:
ListaEnlazada()
void insertarAtras(const T&valor)
void insertarFrente (const T&valor)
void removerAtras()
void removerFrente()
int getLongitud()const
void limpiar()
void insertar(int pos, const T& valor)
void eliminar(int pos)
T& getValor(int pos)
const T& verValor(int indice) const
Nodo<T>* getCabeza() const
T popFront()
~ListaEnlazada()

Implementación Privada:
Nodo<T> * cabeza
Nodo<T> * cola
int longitud
Nodo<T>* getNode(int pos) const

Nodo

Esta es una de las principales clases, es genérica y permite almacenar datos para poder manejar la lógica de las listas enlazadas, sobre todo ya que estas listas poseían un doble enlace.

Interfaz Pública:
Nodo(const T&valor)
T& getDato()
Nodo<T>* getSiguiente() const
Nodo<T>* getAnterior() const
void setSiguiente(Nodo<T>* sig)
void setAnterior(Nodo<T>* ant)
~Nodo()

Implementación Privada:
T dato
Nodo<T> * siguiente
int longitud
Nodo<T> * anterior

TAD'S DE CARACTERÍSTICAS DE LA PARTIDA

Estas son clases que permiten dar características y atributos constantes que en base a ellos se definen comportamientos.

ConfiguracionPartida

Interfaz Pública:
ConfiguracionPartida(DatosConfiguracion* &config)
bool esStacking()const
bool esRetoMas4()const
bool esModoRobo()const
bool esGritoUno()const
bool esGanarNegra()const
bool esFlip()const
~ConfiguracionPartida()

Implementación Privada:
bool stacking
bool retoMas4
bool modoRobo
bool gritoUno
bool ganarNegra
bool flip

ColorCarta

Interfaz Pública:
ColorCarta(const TipoColor &_color)
enum class TipoColor
void setColor(const TipoColor &_color)
TipoColor getColorCarta()
bool operator==(const ColorCarta& otro) const
~ColorCarta()

Implementación Privada:
TipoColor color

Carta

Interfaz Pública:
Carta(Modelo *_reverso, Modelo *_anverso,int _indice)
Carta()
Modelo* getReverso() const
Modelo* getAnverso() const
int getIndice() const
~Carta() = default

Implementación Privada:
Modelo* reverso
Modelo* anverso
int indice

Jugador

Esta es la clase principal que define a cada jugador que participa en una partida.

Interfaz Pública:
Jugador(const std::string _nombre, int codigo)
Jugador(const Jugador& otro)
~Jugador()
bool operator==(const Jugador& otro) const
int getCodigo()const
int getIndice()const
std::string getNombre()const
ListaEnlazada<Carta> * getMazo()
TipoCarta getTipoObligado()
bool getEstaObligado()
void setTipoObligado(TipoCarta tipo)
void setEstaObligado(bool flag)
TipoColor getColorObligado()
bool getObligadoSacar()
void setColorObligado(TipoColor tipo)
void setObligadoSacar(bool flag)
void ordenarCartas(bool estaEnFlip)
int prioridadColor(const Carta& carta,bool estaEnFlip)
bool menorQue(const Carta& a, const Carta& b,bool estaEnFlip)
int prioridadJerarquia(const Carta& carta,bool estaEnFlip)
void desordenarCartas()
std::string saberColorObligado(TipoColor colorCarta)
bool faltaUna()
void evaluarPropensoUno()

bool estaPropensoUno()
void setDijoUno(bool flag)
bool getDijoUno()
bool getObligadoRetar()
void setObligadoRetar(bool flag)

Implementación Privada:
int codigo
std::string nombre
ListaEnlazada<Carta> *mazo
TipoCarta tipoObligado
bool estaObligado
bool obligadoSacar
TipoColor colorObligado
bool propensoUno
bool dijoUno
bool obligadoRetar

CartaDeckUI

Esta es una clase especial heredada de QObject que permite generar las cartas que se muestran en pantalla y estas pueden tener un tamaño y una imagen.

Interfaz Pública:
CartaDeckUI(int _idCarta, const QString _rutaImg, QGraphicsItem *parent = nullptr)
void mousePressEvent(QGraphicsSceneMouseEvent *event) override
void hoverEnterEvent(QGraphicsSceneHoverEvent *event) override
void hoverLeaveEvent(QGraphicsSceneHoverEvent *event) override
~CartaDeckUI()

Implementación Privada:
int indice

Implementación de Signals:
void cartaClickeda(int _indice)

ResultadoJugada

Este es un TAD informal que permite evaluar los resultados que ocurren en cada jugada.

Interfaz Pública:
bool jugadaValida = false
bool darMensaje = false
std::string mensajeJugador
int tiempoMensaje = 0
int tiempoAnimacion = 0
bool requiereDecision = false
std::string colorAviso
bool analizarStack = false
bool esEspia = false

TAD principal

Este es el TAD más grande llamado Partida y es el que se encarga por completo de toda la lógica principal de backend y condiciones que se deben generar para los comportamientos que tiene cada carta.

Interfaz Pública:
Partida(int _cantidadJugadores,DatosConfiguracion* &config)
Jugador* & getJugadorActual()
std::string getDireccion()
int getCantidadPila()
int getCantidadVueltas()
std::string imagenPilaCentral()
std::string imagenPilaLateral()
void verificarVuelta()
void ejecutarMovimiento()
ResultadoJugada ejecutarTirada(int indice)
ResultadoJugada ejecutarDecision(int indice, int decision)
bool getPuedeMoverse()
void setPuedeMoverse(bool accion)
bool getEstaFlip()
void setEstaFlip(bool accion)
bool esUltimaCarta(Jugador * jugadorActual)
Pila<Carta> * getPilaStack()
Pila<Carta> * getPilaLateral()

ConfiguracionPartida getConfiguracion()
ResultadoJugada aplicarAcumuladas()
bool tieneCartasSumaStacking()
bool tieneCartasMulticolorStacking()
void llenarPilaLateral()
std::string getNombreColor()
void setPuedeRetar(bool flag)
bool getPuedeRetar()
void setColorJuegoSelect(ResultadoJugada &resultadoTirada, int decision)
void setVecesSumadasCarta(int veces)
int getVecesSumadasCarta()
void setEstaEclipse(bool flag)
bool getEstaEclipse()
void ordenCartas()
void desordenarCartas()
Jugador * pickJugadorSiguiente()
Jugador * pickJugadorAnterior()
bool alguienEnUno()
std::string gritarUno(std::string texto)
void penalizarJugador(Jugador * &jugadorParam)
void verificarPilaLateral()
void obligarJugadorRetar()
std::string ejecutarReto()
Jugador * jugadorSumar(bool &movio)
ListaCircular<Jugador*>& getListaJugadoresPartida()
~Partida()

Implementación Privada:
ListaCircular<Jugador*> listaJugadores
ConfiguracionPartida configuracion
int cantidadJugadores
std::string direccion
int cantidadVueltas
bool puedeMoverse
bool puedeRetar
bool estaFlip
std::string colorPartida
bool estaEclipse
int vecesSumadasCarta
Modelo * modelosClaros[60]
Modelo * modelosOscuros[60]
ListaEnlazada<Carta> * listadoCartas ListaEnlazada<Carta> * listaCartasBlancas
ListaEnlazada<Carta> * listadoCartasOscuras
Pila<Carta> *pilaLateralCartas
Pila<Carta> *pilaCentralCartas
Pila<Carta> * pilaStacking
Nodo<Jugador*> inicioRonda = nullptr
void generarJugadores()
int generarIndiceRandom(int limiteSuperior)
void correrIndices()
void generarDireccionInicial()
void generarCartasClaros()
void generarCartasOscuras()

void armarCartas()
void armarCartasNormal(ListaEnlazada<Carta>* & lista)
void armarCartasFlip(ListaEnlazada<Carta>* & lista)
void armarCartasOscuras(ListaEnlazada<Carta>* & lista)
void barajarCartas(ListaEnlazada<Carta> * & lista)
void limpiarReferencias()
void repartirCartas()
bool tieneEnClaros()
bool tieneEnOscuras()
bool puedeGanarConNegra(int indice)
bool esMismaClara(const Carta& cartaJugador, const Carta& cartaPila)
bool esMismaOscura(const Carta& cartaJugador, const Carta& cartaPila)
ResultadoJugada ejecutarAccionCartaClara(int indice)
ResultadoJugada ejecutarAccionCartaOscura(int indice)
ResultadoJugada accionCartaEspecialClara(int indice)
ResultadoJugada accionCartaEspecialOscura(int indice)
void establecerColorPartida(TipoColor colorCarta)
TipoColor getColorPartida()
void definirColorPrincipal()
TipoColor generarColorAleatorio()
ResultadoJugada tirarCartaNumericaClara(Carta& cartaElegida, int indice)
ResultadoJugada tirarBloqueoClara(Carta& cartaElegida, int indice)
ResultadoJugada tirarCambioDireccionClara(Carta& cartaElegida, int indice)
ResultadoJugada tirarSumaClara(Carta& cartaElegida, int indice)
ResultadoJugada tirarCartaNumericaOscura(Carta& cartaElegida, int indice)
ResultadoJugada tirarSaltoOscura(Carta& cartaElegida, int indice)
ResultadoJugada tirarCambioDireccionOscura(Carta& cartaElegida, int indice)

ResultadoJugada tirarSumaOscura(Carta& cartaElegida, int indice)
ResultadoJugada tirarCartaComodinClara(Carta& cartaElegida, int decision, int indice)
ResultadoJugada tirarCartaSumaComodinClara(Carta& cartaElegida, int decision,int indice)
ResultadoJugada tirarCartaSumaComodinOscura(Carta& cartaElegida, int decision,int indice)
ResultadoJugada tirarCartaEternaOscura(Carta& cartaElegida, int decision,int indice)
ResultadoJugada tirarCartaEclipse(Carta& cartaElegida, int indice)
ResultadoJugada tirarCartaEspia(Carta& cartaElegida, int indice)
ResultadoJugada tirarCartaReversa(Carta& cartaElegida, int indice, bool adelante)
void ejecutarPrimerMovimiento()
void evaluarObligacionColor(ResultadoJugada &resultado, Carta &cartaTomada)
TipoColor getColorDecision(int decision)

TAD'S DE LA UI

Debido a que se trabajó con una interfaz gráfica esta utiliza el mismo principio de dividir por TAD 'S cada screen por lo tanto se adjunta todo lo utilizado de lógica de UI.

MainWindow

Interfaz Pública:
MainWindow(QWidget *parent = nullptr)
~MainWindow()

Implementación Privada:
Ui::MainWindow *ui
bool estaConfigurando
PantallaInicio *inicio = nullptr
PantallaSeleccion * pantallaSelect = nullptr
PantallaModalidad * pantallaModal = nullptr
PantallaConfiguraciones * pantallaConfiguracion = nullptr
PantallaJuego * juegoScreen = nullptr
DatosConfiguracion * datosConfig = nullptr
void mostrarSeleccion()
void mostrarModalidad(int cantidadPersonas, bool personalizacion)
void mostrarConfiguraciones()

void mostrarJuego(int _cantidadPersonas)
void vaciarPunteroDatos()

PantallaInicio

Interfaz Pública:
PantallaInicio(QWidget *parent = nullptr)
~PantallaInicio()

Implementación Privada:
void on_pushButton_2_clicked()
void on_btnInicio_clicked()

Implementación de Signals:
void solicitarSeleccion()

PantallaSeleccion

Interfaz Pública:
PantallaSeleccion(QWidget *parent = nullptr)
~PantallaSeleccion()

Implementación Privada:
Ui::PantallaSeleccion *ui
void on_btnRegreso_clicked()
void on_btnDosJugadores_clicked()
void on_btnTresJugadores_clicked()
void on_btnCuatroJugadores_clicked()
void on_btnMasJugadores_clicked()

Implementación de Signals:
void solicitarRegresoInicio()
void solicitarModalidad(int cantidadPersonas, bool personalizacion)

PantallaJuego

Interfaz Pública:
PantallaInicio(QWidget *parent = nullptr)
~PantallaInicio()
void onCartaPresionada(int indice)

Implementación Privada:
Ui::PantallaJuego *ui
QGraphicsScene *escena
QGraphicsView *vista
QTimer* timerMensaje
QWidget * capaBloqueo
PartidaController * controladorPartida = nullptr
int cantidadJugadores
void mostrarDatosPantalla(Jugador* & jugadorActual, std::string direccion, int cartasPila, int vueltas, bool verificar, std::string colorPartida)
void inicializarVistaMazo()
void dibujarMazo(Jugador* & jugadorActual, bool verificar)
void prepararInterfazCartas()

void actualizarPilaCentral(std::string rutaImagen)
void actualizarBarajaLateral(std::string rutaImagenPila, int longitudPila)
void darMensajeJugador(std::string mensaje, QString colorHex, int tiempo)
void instanciarTimer()
void verificarStacking(bool evaluar)
void ejecutarSumaCartas()
bool mostrarConfirmacionStacking(std::string mensaje)
int mostrarSelectorColor(bool estaFlip)
void evaluarJugadoresEnUno()
void mostrarMazoTemporal()
void analizarGanador()
bool analizarReto()
void on_btnRetorno_clicked()
void on_btnPilaLateral_clicked()
void on_btnUNO_clicked()

Implementación de Signals:
void solicitarSalida()

PantallaConfiguraciones

Interfaz Pública:
PantallaConfiguraciones(bool &flag, DatosConfiguracion * &config, QWidget *parent = nullptr)
void setChekedOpciones(DatosConfiguracion * &config, bool & flag)
~PantallaConfiguraciones()

Implementación Privada:
Ui::PantallaSeleccion *ui
DatosConfiguracion * a_config = nullptr
void on_btnRetorno_clicked()
void on_checkRobo_toggled(bool checked)
void on_checkGrito_toggled(bool checked)
void on_checkGanarNegra_toggled(bool checked)
void on_checkBoxFlip_toggled(bool checked)

Implementación de Signals:
void solicitarRegresoModalidad()
void solicitarModalidad(int cantidadPersonas, bool personalizacion)

SelectorColorDialog

Interfaz Pública:
SelectorColorDialog(bool _estaFlip, QWidget *parent = nullptr)
void aplicarEstilos()
void configurarColores(bool estaFlip)
int getColorElegido() const
~SelectorColorDialog()

Implementación Privada:
Ui::SelectorColorDialog *ui
bool estaFlip
int colorElegido
void on_btnColor1_clicked()
void on_btnColor2_clicked()
void on_btnColor3_clicked()
void on_btnColor4_clicked()

VerMazoDialog

Interfaz Pública:
explicit VerMazoDialog(ListaCircular<Jugador*> &jugadores, bool modoFlip, QWidget *parent = nullptr)
~VerMazoDialog()

Implementación Privada:
Ui::SelectorColorDialog *ui
ListaCircular<Jugador*> &listaJugadores
bool estaFlip
QGraphicsScene* escena
QGraphicsView* vista
int jugadorSeleccionado
void inicializarVistaMazo()
void cargarJugadores()
void mostrarMazoJugador(int indice)
void dibujarMazo(Jugador* jugadorActual)
void setReferenciaMazo()

PartidaController

Esta es una de las clases principales del proyecto ya que en el proyecto se utilizó un pseudo modelo vista controlador en donde el backend es independiente al frontend por lo tanto se hizo este tipo de implementación para darle escalabilidad.

Interfaz Pública:
explicit PartidaController(int _cantidad, bool &estaConfigurando, DatosConfiguracion * &config, QObject* parent = nullptr)
ResultadoJugada tirarCarta(int indice)
bool estaModoFlip()
bool puedeDesapilar()
bool estaStackeando()
ResultadoJugada aplicarCartasStackeadas()
bool puedeStackear()
bool tieneParaStackear()
ResultadoJugada desapilarCarta()
std::string getMensajeStacking()
void obligarJugador()
ResultadoJugada decisionJugador(int indice, int decision)
void setearColorPartida(int decision)
bool getEstaEclipse()
bool permiteFlip()
bool jugadorPropensoUno()
bool alguienEstaEnUno()
void gritarUno(QString mensaje)
void refrescarEvaluacionUno()
ListaCircular<Jugador*> &getListaJugadores()
void aplicarMovimiento()

bool jugadorGano()
void darMensajeGanador()
bool puedeRetar()
bool estaStackeandoComodin()
void obligarRetar()
void retarJugador()
~PartidaController()

Implementación Privada:
Partida * gestorPartida

Implementación de Signals:
void datosPartida(Jugador* &jugador, std::string direccion, int cartasPila, int vueltas, bool verificar, std::string colorPartida)
void datosPilaCentral(std::string direccion)
void datosPilaLateral(std::string direccion, int cartasPila)
void darMensaje(std::string mensaje, QString colorHex, int tiempo)